

Análisis de Lenguajes y Compiladores

Paradigmas, Rendimiento y Diseño de DSL (ECO-GRID)

Universidad Nacional Experimental de Guayana (UNEG)

Prof. Félix Márquez | Kernel 4

Actividad I

Análisis Paradigmático y Matriz Descriptiva

Evolución y Convergencia Paradigmática



La Necesidad Multiparadigma

La industria actual exige enfoques mixtos. Se utiliza la **Orientación a Objetos** para estructurar y modelar la lógica del negocio, mientras que se adoptan rasgos del paradigma **Funcional** para transformar datos en memoria de forma declarativa y libre de errores.



Impacto en el AST (Árbol Sintáctico)

Las decisiones léxicas no son simples estilos. En entornos **imperativos**, el AST es denso en bucles y mutaciones de memoria. En entornos **funcionales**, el AST es un grafo de expresiones matemáticas que permite al compilador paralelizar procesos automáticamente.

Matriz Comparativa de Paradigmas

Paradigma	Arquitectura Lógica	Tratamiento de Memoria
Imperativo	Abstracción de la Máquina de Von Neumann.	Mutabilidad inherente y reasignación continua.
Orientado a Objetos	Modelado mediante cohesión de estado/comportamiento.	Estado controlado a través de interfaces (métodos).
Funcional	Cálculo lambda y funciones de orden superior.	Inmutabilidad estricta y transparencia referencial.
Concurrente	Diseño para arquitecturas de múltiples núcleos.	Aislamiento estricto (cero memoria compartida).

Actividad II

Estudio Morfológico, Sintáctico y Benchmarking (Collatz)

Análisis Morfológico y Sintáctico



Python (Interpretado)

Espacios significativos. El escáner inyecta tokens ocultos `INDENT` y `DEDENT` para definir bloques lógicos, eliminando el uso de llaves.



JavaScript (Compilación JIT)

Escáner altamente permisivo. Destaca su mecanismo ASI (*Automatic Semicolon Insertion*), donde el motor infiere e inyecta delimitadores faltantes.



Rust (Nativo LLVM)

Diseñado para seguridad. Léxico que define inmutabilidad por defecto. Sintaxis limpia que omite paréntesis en las expresiones booleanas.



Zig (Nativo LLVM)

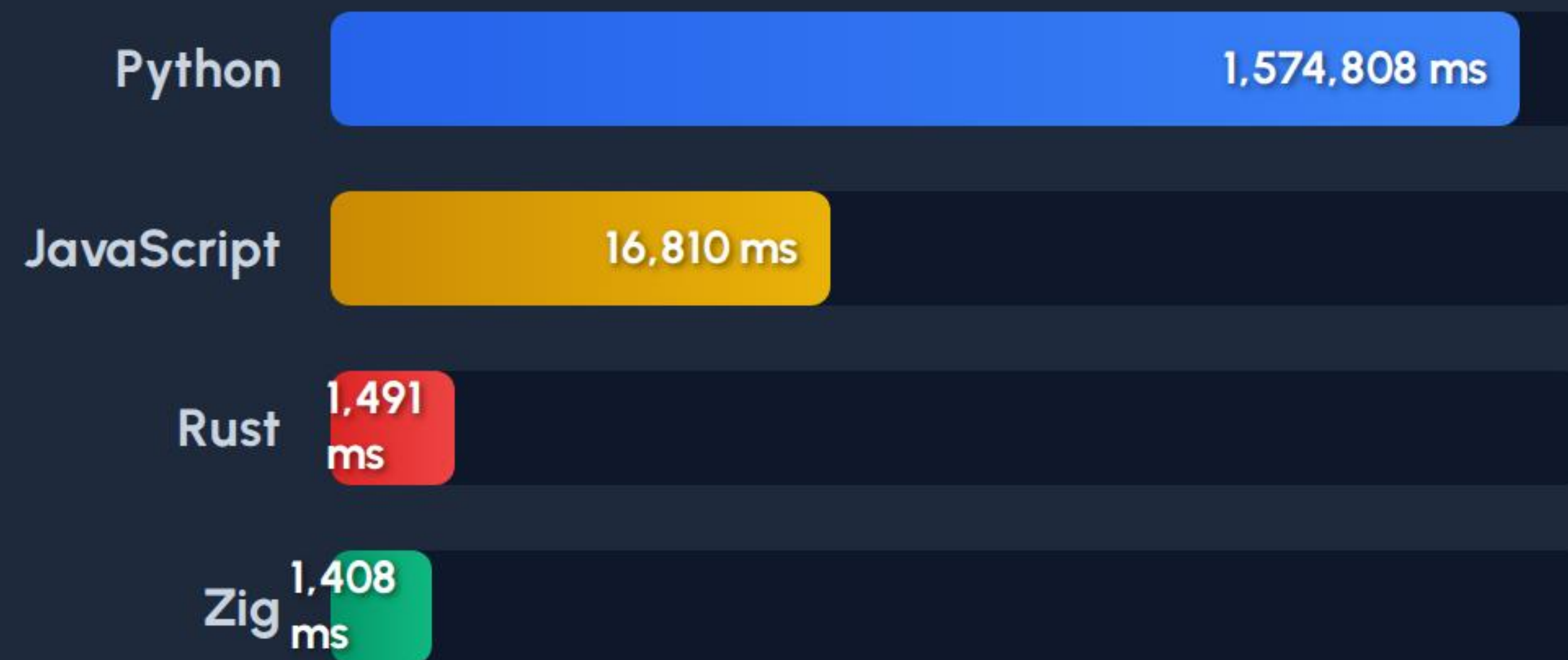
Analizador estricto y minimalista. Palabras reservadas limitadas y dependencia total de delimitadores explícitos (`{}` y `;`).

Benchmarking Intensivo (10 Millones de Iteraciones)

Lenguaje (Paradigma)	Mecanismo de Ejecución	Tiempo Total (ms)	RAM Pico OS (MB)
Zig (Imperativo)	Compilación Nativa (LLVM)	1,408.49	3.04
Rust (Multiparadigma)	Compilación Nativa (LLVM)	1,491.01	4.31
JavaScript (Multi.)	Compilación JIT (Node.js)	16,810.30	39.05
Python (Multi.)	Interpretado Puro (CPython)	1,574,808.59	11.89

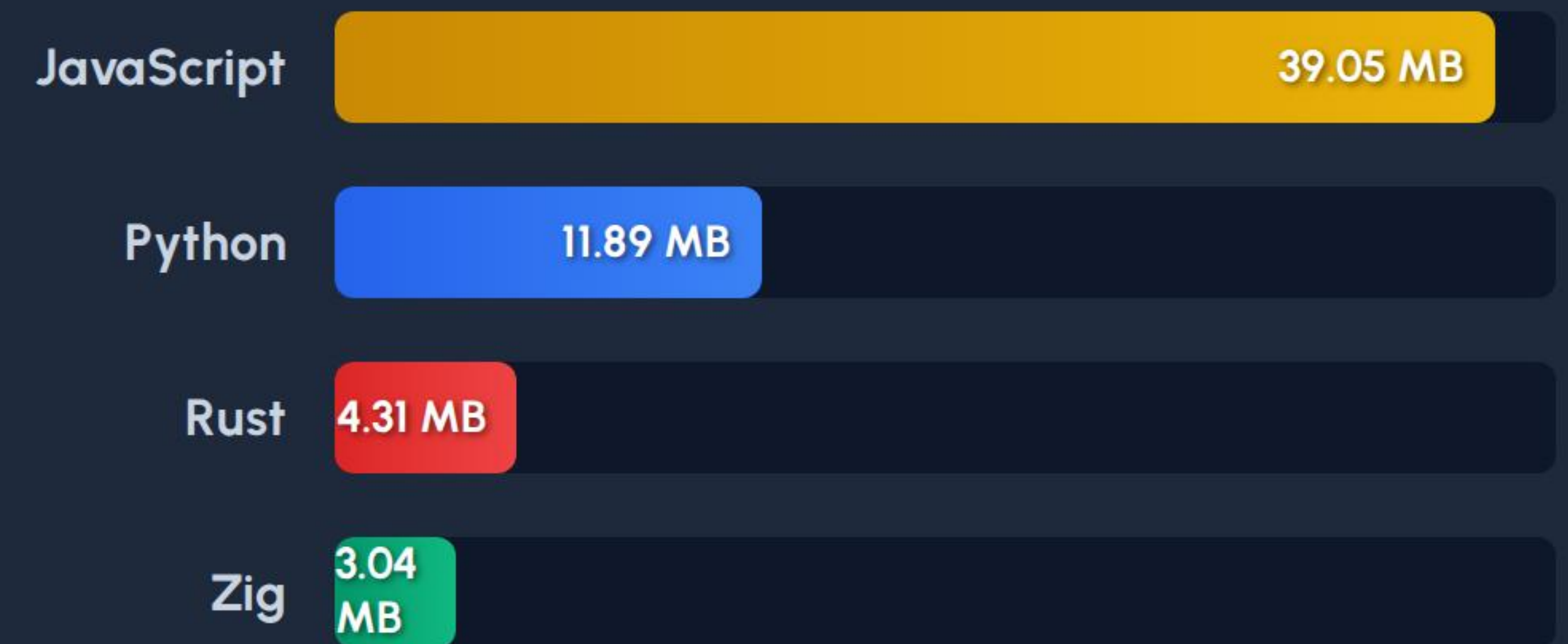
Comparativa de Rendimiento Arquitectónico

Tiempo de Ejecución (ms)



*Escala visual logarítmica por diferencia extrema.

Consumo RAM Pico OS (MB)



*JS (JIT) sacrifica RAM por velocidad frente a Python.

Actividad III

Diseño de Lenguaje de Dominio Específico (DSL ECO-GRID)

Diseño Léxico (ECO-GRID)



Objetivo del DSL

Abstraer el control físico de relés y sensores en microredes eléctricas para que los operadores industriales puedan configurar reglas sin cometer errores de sintaxis.



Legibilidad Extrema

Prescinde de delimitadores estrictos de cierre de línea (como el punto y coma) y adopta un vocabulario nativo en español para comandos lógicos y primitivas.

Clasificación de Tokens del Escáner

LIT_NUM

Literales numéricos (ej. `55.5`, `100`).

LIT_BOOL

Booleanos nativos (`encendido`, `apagado`).

LIT_NIVEL

Criticidad (`info`, `critico`).

ID

Identificadores de usuario en convención `camelCase`.

ESTRUCTURA

Palabras de flujo en MAYÚSCULAS (`SI`, `MIENTRAS`).

PRIMITIVA

Funciones nativas en `snake_case`.

OP

Operadores lógicos y matemáticos (`+`, `Y`, `0`).

DELIM

Paréntesis `()`, corchetes `[]` y comas.

COMENTARIO

Líneas descartadas precedidas por el símbolo `#`.

Gramática Abstracta

-  **Asignación:** Estructura lineal que exige un ID, el operador = y finaliza con salto de línea.
-  **Condicionales:** Bloques regulados por SI (condición) ENTONCES y finalizados con FIN_SI.
-  **Iterativas:** Ciclos MIENTRAS para monitorización y PARA_CADA para recorrer agrupaciones de hardware.
-  **Resiliencia:** Uso obligatorio de INTENTAR y EN_CASO_DE_FALLA para asegurar el sistema ante la pérdida de conexión con sensores.

Escenario A: Prevención de Fuga Térmica

Objetivo: Monitorización iterativa para prevenir el sobrecalentamiento de las celdas. Si la temperatura supera 55.0°C, aísla la carga, activa refrigeración y aborta el ciclo para emitir alerta.

```
init_grid()
monitoreoTermico = encendido

MIENTRAS (monitoreoTermico == encendido) EJECUTAR
    INTENTAR
        tempActual = leer_temperatura(bateriaPrincipal)

        SI (tempActual > 55.0) ENTONCES
            # 1. Activar refrigeracion y aislar calor
            activar_refrigeracion(bateriaPrincipal)
            conmutar_linea(sensorPaneles1, apagado)

            # 2. Desviar el consumo industrial
            conmutar_linea(sectorIndustrial, apagado)
            conmutar_linea(redComercial, encendido)
```

```
        # 3. Periodo de verificacion
        esperar(10)
        tempVerificacion = leer_temperatura(bateriaPrincipal)

        SI (tempVerificacion > 55.0) ENTONCES
            emitir_alerta(critico, "Peligro persistente de
fuga. Exceso: " + (tempVerificacion - 55.0))
            ROMPER
        FIN_SI
    FIN_SI
    EN_CASO_DE_FALLA
        emitir_alerta(critico, "Fallo en sensor termico: " +
bateriaPrincipal)
    FIN_INTENTAR

    esperar(60)
FIN_MIENTRAS
```

Escenario B: Balance de Carga y Optimización

Objetivo: Decisiones autónomas. El sistema inyecta superávit a la red pública o aísla cargas pesadas de noche para preservar el suministro en los servidores médicos.

```
init_grid()
horaMilitar = hora_actual()

INTENTAR
    cargaActual = estado_carga(bateriaPrincipal)
    flujoSolar = flujo_actual(sensorPaneles1)
    demandaInterna = flujo_actual(sensorConsumoInterno)

    # CASO 1: Inyección/Venta de excedentes
    SI (cargaActual > 90 Y flujoSolar > demandaInterna) ENTONCES
        conmutar_linea(redPublica, encendido)
        emitir_alerta(info, "Inyectando excedente a red publica.
Potencia: " + (flujoSolar - demandaInterna))
    FIN_SI
```

```
    # CASO 2: Aislamiento preventivo nocturno
    SI (cargaActual < 20 Y horaMilitar ≥ 18 Y horaMilitar ≤ 23)
ENTONCES
    # Desconectar cargas pesadas no esenciales
    conmutar_linea(sectorIndustrial, apagado)

    # Asegurar flujo eléctrico a áreas críticas
    conmutar_linea(sectorMedico, encendido)
    conmutar_linea(sectorServidores, encendido)

    emitir_alerta(advertencia, "Racionamiento activo.
Bateria: " + cargaActual)
    FIN_SI
EN_CASO_DE_FALLA
    emitir_alerta(critico, "Error en bus de datos.")
FIN_INTENTAR
```

¿Preguntas?

Gracias por su atención

